

Análise de Código Detalhada: Código por Código

Mapeamento explícito linha por linha do funcionamento da Urna Eletrônica

Este documento oferece um mapeamento técnico rigoroso, analisando individualmente cada bloco de código estrutural fornecido no sistema de votação Java. A tabela a seguir divide os fragmentos do programa, suas responsabilidades computacionais e os conceitos envolvidos.

1. Módulos de Entrada e Entidades POO

Bloco de Código / Linha	Explicação Funcional e Técnica
<pre>import java.util.ArrayList; import java.util.Scanner;</pre>	Importações obrigatórias da API nativa do Java. O ArrayList é usado para gerenciar listas dinâmicas em memória e o Scanner serve para capturar as entradas digitadas pelo usuário no terminal.
<pre>abstract class Pessoa { private String nome; private String cpf;</pre>	Definição da classe base como abstrata , impedindo que seja instanciada de forma isolada. Os atributos possuem o modificador private, aplicando o pilar do Encapsulamento para proteger os dados sensíveis de modificações externas diretas.
<pre>public Pessoa(String nome, String cpf) { this.nome = nome; this.cpf = cpf; }</pre>	Método construtor da classe Pessoa. Ele recebe os parâmetros iniciais no momento da criação e utiliza a palavra-chave this para diferenciar as variáveis locais dos atributos do objeto instanciado.
<pre>public String getNome() { return nome; } public void setNome(String nome) { ... }</pre>	Métodos acessores públicos (Getters e Setters). Permitem a leitura e alteração segura e controlada das variáveis privadas por outras partes do software.
<pre>class Eleitor extends Pessoa { private final String celular; private final String endereco; private final int voto;</pre>	Aplicação do conceito de Herança (extends). A subclasse Eleitor herda os atributos de Pessoa e adiciona dados exclusivos da eleição. O termo final indica que estes valores não sofrerão alterações após definidos no construtor.
<pre>public Eleitor(...) { super(nome, cpf); this.celular = celular; ... }</pre>	O construtor da classe filha aciona obrigatoriamente o construtor da classe pai usando a instrução super(nome, cpf), garantindo o reaproveitamento lógico dos campos básicos de identificação antes de setar as novas propriedades.

2. Inicialização e Estrutura de Controle (Main)

Bloco de Código / Linha	Explicação Funcional e Técnica
<pre>try (Scanner scanner = new Scanner(System.in)) {</pre>	Abertura do fluxo do teclado usando a sintaxe Try-with-resources . Garante que, independentemente de falhas ou término precoce, o recurso do sistema operacional (System.in) seja desalocado automaticamente ao fim do bloco.
<pre>ArrayList<Eleitor> listaEleitores = new ArrayList<>(); String[] candidatos = {"Ana...", ...}; int[] votosContagem = new int[3];</pre>	Configuração da memória volátil. Define a coleção dinâmica de eleitores armazenados, uma matriz (array fixo) de Strings contendo o rol nominal dos candidatos, e um vetor matemático de inteiros de tamanho 3 para somar e agrupar os votos recebidos.
<pre>final String LOGIN_ADMIN = "1234"; final String SENHA_ADMIN = "1234";</pre>	Definição de constantes estáticas em nível de método local utilizando o modificador final. Elas servem de parâmetro rígido e imutável para validar o acesso às áreas administrativas.
<pre>while (sistemaAtivo) { ... }</pre>	Laço de repetição condicional principal. Mantém o sistema de votação em execução e o menu aberto em loop enquanto a variável booleana sistemaAtivo for verdadeira.

3. Fluxo de Validação de Dados (Opção 1)

Bloco de Código / Linha	Explicação Funcional e Técnica
<pre>while (true) { nome = scanner.nextLine(); if (nome.length() >= 3) break; ... }</pre>	Loop contínuo de consistência de dados. Captura o nome completo digitado e só quebra o laço (break) se a string atender ao requisito de segurança mínima de conter 3 caracteres ou mais.
<pre>if (cpf.length() != 11) { continue; }</pre>	Garante a integridade do CPF rejeitando qualquer digitação que possua tamanho diferente de 11 algarismos, saltando o restante do código usando continue para reiniciar a solicitação do campo.
<pre>boolean cpfDuplicado = false; for (Eleitor e : listaEleitores) { if (e.getCpf().equals(cpf)) { ... } }</pre>	Algoritmo de varredura contra fraude eleitoral. Utiliza o laço aprimorado for-each para comparar o CPF recém-digitado com todos os CPFs guardados na listaEleitores. Identifica duplicidades usando comparação de conteúdo de String (.equals()).
<pre>try { int voto = Integer.parseInt(...); if (voto >= 1 && voto <= 3) { votosContagem[voto - 1]++; } ... }</pre>	O programa lê o número do voto do terminal e tenta convertê-lo de texto para um tipo numérico inteiro. Se o valor estiver no intervalo de 1 a 3, incrementa em 1 a contagem da posição correta do vetor (aplicando deslocamento de índice zero com voto - 1).
<pre>catch (NumberFormatException e) { ... listaEleitores.add(new Eleitor(..., 0)); }</pre>	Tratamento de exceções robusto. Caso o eleitor digite letras ou caracteres inválidos no campo numérico do voto, o bloco intercepta a falha de conversão (NumberFormatException), impede a interrupção do programa, computa o voto como valor zero (Nulo) e registra o eleitor com segurança.

4. Auditoria, Encerramento e Apuração (Opções 2, 3 e Boletim)

Bloco de Código / Linha	Explicação Funcional e Técnica
<pre>if (user.equals(LOGIN_ADMIN) && ...)</pre>	Estrutura condicional lógica de controle de acesso. Concede acesso ao painel do administrador apenas se o par de credenciais fornecido corresponder exatamente às constantes imutáveis gravadas no sistema.
<pre>for (Eleitor e : listaEleitores) { String vStr = (e.getVoto() >= 1 && ...) ? candidatos[e.getVoto()-1] : "Nulo";</pre>	Rotina de Auditoria de Dados . Varre os objetos da lista e usa um operador ternário (condição) ? verdadeiro : falso para verificar o número do voto, resgatar o nome do respectivo candidato no vetor nominal ou classificá-lo textualmente como

Bloco de Código / Linha	Explicação Funcional e Técnica
<pre>... }</pre>	"Nulo". Em seguida, imprime os dados completos de identificação do eleitor.
<pre>case "2" -> { sistemaAtivo = false; exibirBoletim = true; }</pre>	Encerramento programático padrão. Desliga o laço while alterando o estado de sistemaAtivo para falso e mantém ativada a exibição dos resultados consolidados da urna na saída de tela.
<pre>else if (opcaoPrincipal.equals("3")) { ... exibirBoletim = false; }</pre>	Tratamento para a opção de saída rápida do menu raiz. Exige autenticação administrativa prévia e, se bem-sucedida, desliga o loop global marcando a flag exibirBoletim = false, o que oculta intencionalmente o relatório final de votos (fechamento emergencial).
<pre>if (exibirBoletim) { double porc = (totalVotos > 0) ? ((double) votosContagem[i] / totalVotos) * 100 : 0; System.out.printf("... %.2f%%\n", porc); }</pre>	Bloco final executado após a quebra dos laços. Calcula o percentual individual de cada candidato convertendo explicitamente o contador de inteiros para ponto flutuante (double), prevenindo erros de divisão por zero. Imprime o Boletim de Urna Final estruturado com o modificador de formatação %.2f para travar a saída em duas casas decimais.